保存到APP查看 印象笔记

Agent Workflow以及4种主流设计模式(吴恩达)

今天分享的是吴恩达教授在美国红杉AI活动上关于Agent的最新趋势与洞察。吴恩达表示Agent的工作流程与传统使用LLM的方式不同，它更多的是迭代和对话式。

目前，有4种主要的Agent设计模式，分别是：

1）反思：让Agent利用并修改自己生成的输出；

2）工具使用：LLM生成代码、调用API等进行实际操作；

3）Planning：让Agent拆分复杂任务并按计划执行；

4）多Agent协作：多个Agent扮演不同角色合作完成任务；

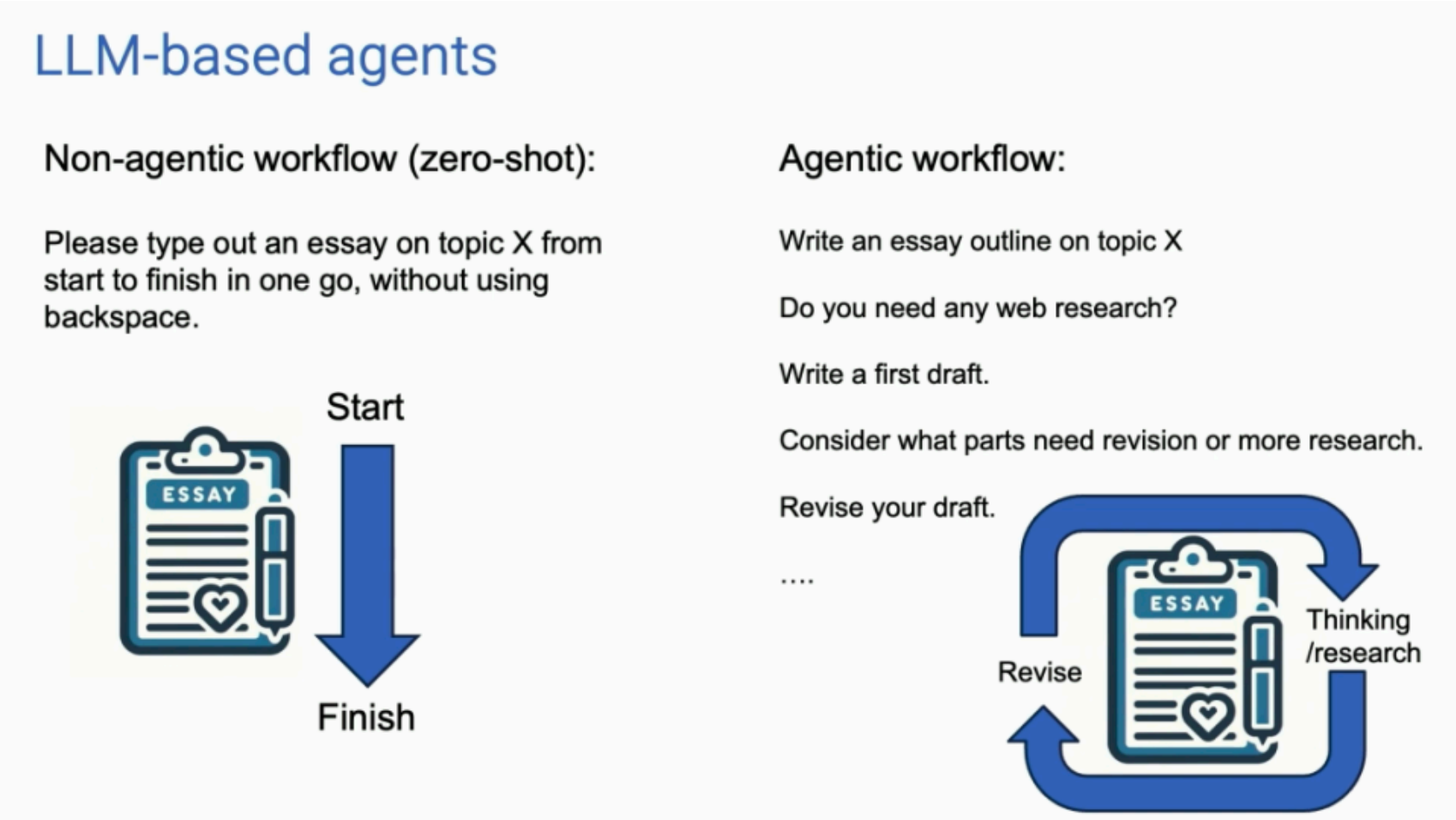
这些设计模式目前仍在快速发展中，有的比较成熟，有的仍存在不确定性，但都落实了提高人工智能能力的潜力。

吴恩达认为，未来AI Agent的能力将大幅扩展，我们需要学会将任务委托给Agent并等待等待结果，而不是追求即时响应。另外，快速代币生成也很重要，即使基于质量较低的LLM，快速迭代生成新代币也可能获得良好的结果。

以下为本次分享的全部内容：

我与大家分享了我对 AI 代理所看到的东西，这是我认为每个人在构建 AI 时都应该注意的激动人心的趋势。同时，我也对所有其他的“接下来会发生什么”的展示感到兴奋。

AI Agent，今天我们大多数人使用LLM的方式就像这样，通过一个非代理性工作流程，你输入一个提示，它生成一个答案。这有点像如果你让一个人就某个话题写一篇论文，我说，请坐到键盘前，从头到尾不断地打字写完这篇论文，从不使用退格键。虽然这样做有多困难，LLM却能做出这样的好。



相比之下，Agent性工作流程可能是这样的：让AI或LLM说，写一篇论文大纲。你需要做任何网络研究吗？如果需要，让我们开始吧。然后写第一篇稿子，然后阅读你自己的第一个稿子，思考哪些部分需要修改，然后修改你的草稿，你可以一遍流程

又一遍地进行。因此，这个工作修改迭代更多，你可能让模型进行一些思考，然后文章，然后再思考，并通过多次迭代来完成这个过程。

Coding benchmark (HumanEval)

Example problem:

```
def solution(lst):
    """Given a non-empty list of integers, return the sum of all of the odd elements
    that are in even positions.

    Examples
    solution([5, 8, 7, 1]) ==>12
    solution([3, 3, 3, 3, 3]) ==>9
    solution([30, 13, 24, 321]) ==>0
    """
```

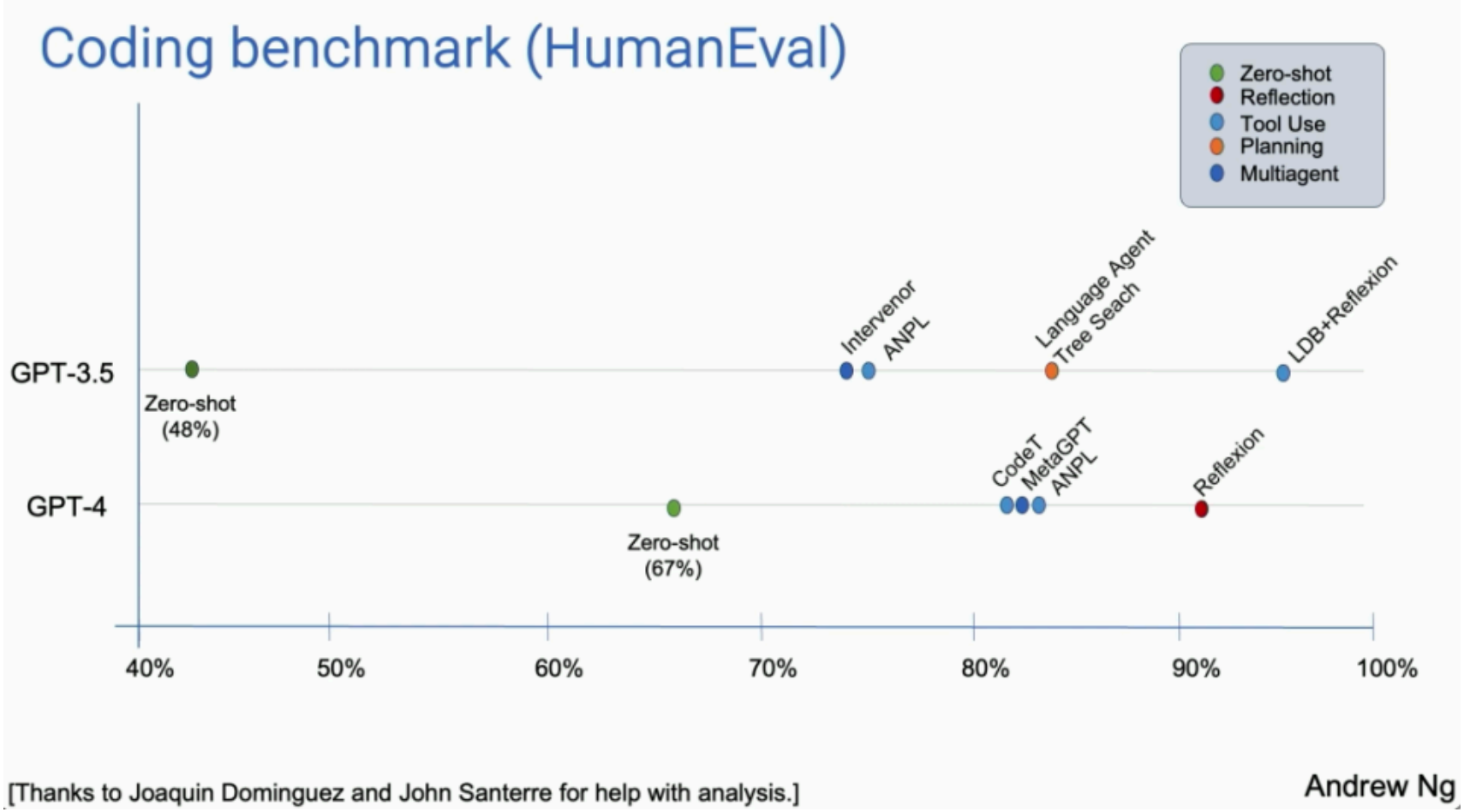
Solution:

```
return sum(lst[i] for i in range(0,len(lst)) if i % 2 == 0 and lst[i] % 2 == 1)
```

Source: Evaluating Large Language Models Trained on Code, Chen et al., 2021

Andrew Ng

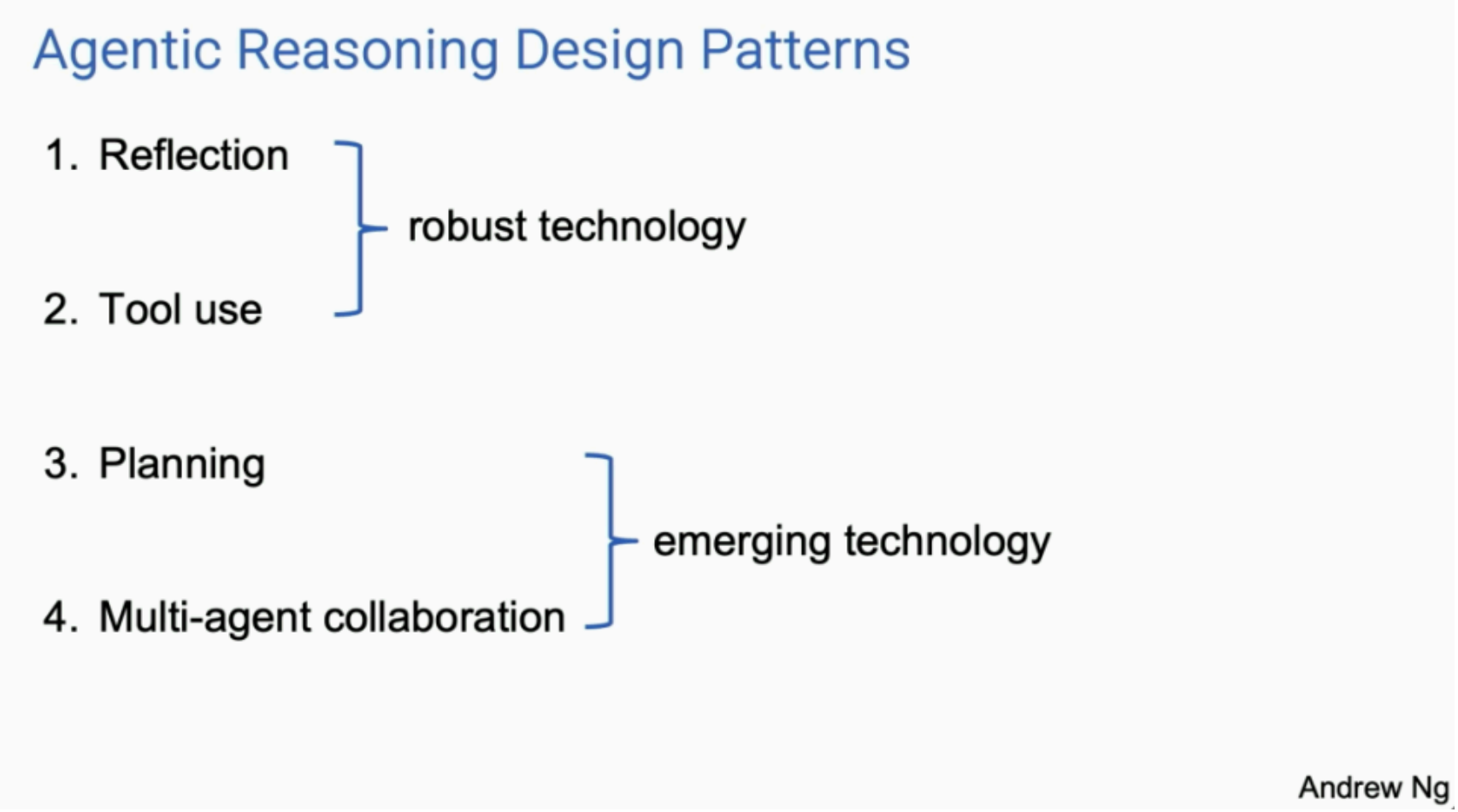
许多人没有意识到，这样可以获得显着更好的结果。实际上，我自己也对对话工作流程的效果非常惊讶。我将在我的团队中做一个案例研究，使用几年前的 OpenAI发布一个名为 Human Eval Benchmark 的编码基准来分析一些数据，但有编码问题，比如给定非负整数列表，返回所有奇数元素或不均匀位置的总和。



结果是，像这样的代码片段。今天我们很多人会使用零次提示，意味着告诉我们人工智能写入代码并在第一次尝试时运行它。说谁会那样编码？没有人会那样编码。我们只是打出代码并运行它。也许你可以。我做不到。结果是，如果你使用GPT-3.5零次提示，它的正确率为48%。

GPT-4做得更好，正确率达到了67.7%，但如果你围绕GPT-3.5使用一个代理性工作流程，实际上它的表现甚至比GPT-4还要好。如果你将这种类型的工作流程评价GPT-4，它也表现得非常好。你会注意到，GPT-3.5与一个代理性工作流程相结合实际上超过了GPT-4的表现。

我认为这意味着，这对我们构建自主应用程序的方式有着第二层次的影响。Agent是一个经常被提及的术语和任务，有很多咨询报告讨论了Agent、AI的未来等等。



我想具体分享一下我在代理方面看到的广泛设计模式。这是一个非常混乱、动荡的领域，有大量的研究和开源项目。发生了很多事情，但我试图将其更具体地归结为事情已经发生了。

1.反思

Agent Reflection是一个工具，我认为我们很多人都在使用，它就是作业工具。我认为它得到了更广泛的认可，而且实际效果相当好。我认为这些是相当健壮的技术。当我使用它们时，我几乎总是让它们运行得很好，规划和多代理合作。

我认为这更多是一个新兴的领域，当我使用它们时，有时我对它们的表现感到惊讶，但至少在目前，我觉得我不能总是可靠地让它们工作。让我通过几个方面来讲解这四种设计模式。如果你们中的一些人重新尝试，或者让你们的工程师使用这些，我想你们很快就会获得生产力的提升。

关于Reflection，这里有一个例子。比方说我让一个系统为我写完成一个给定的任务。然后我们有一个编码代理，只是一个你提示写代码的LLM，也就是说，“嘿，定义doTask，就像这样写一个函数。”

它自我反思的一个例子可能是，如果你然后用类似这样的提示对LLM进行提示：“这里有一段代码是为了完成一个任务，只需把它们刚刚生成的四个代码给，说，然后仔细检查代码的正确性、效率、构建是否良好。只需要像这样写一个提示。”

结果可能是，你提示写代码的同一个LLM，可能能够发现诸如第5行的这个错误，并通过某种方式修复它。如果你现在把它自己的反馈给它，并重新提示它，它可能会输出一个第二版本的代码，这个版本的代码可能比第一个版本工作得更好。

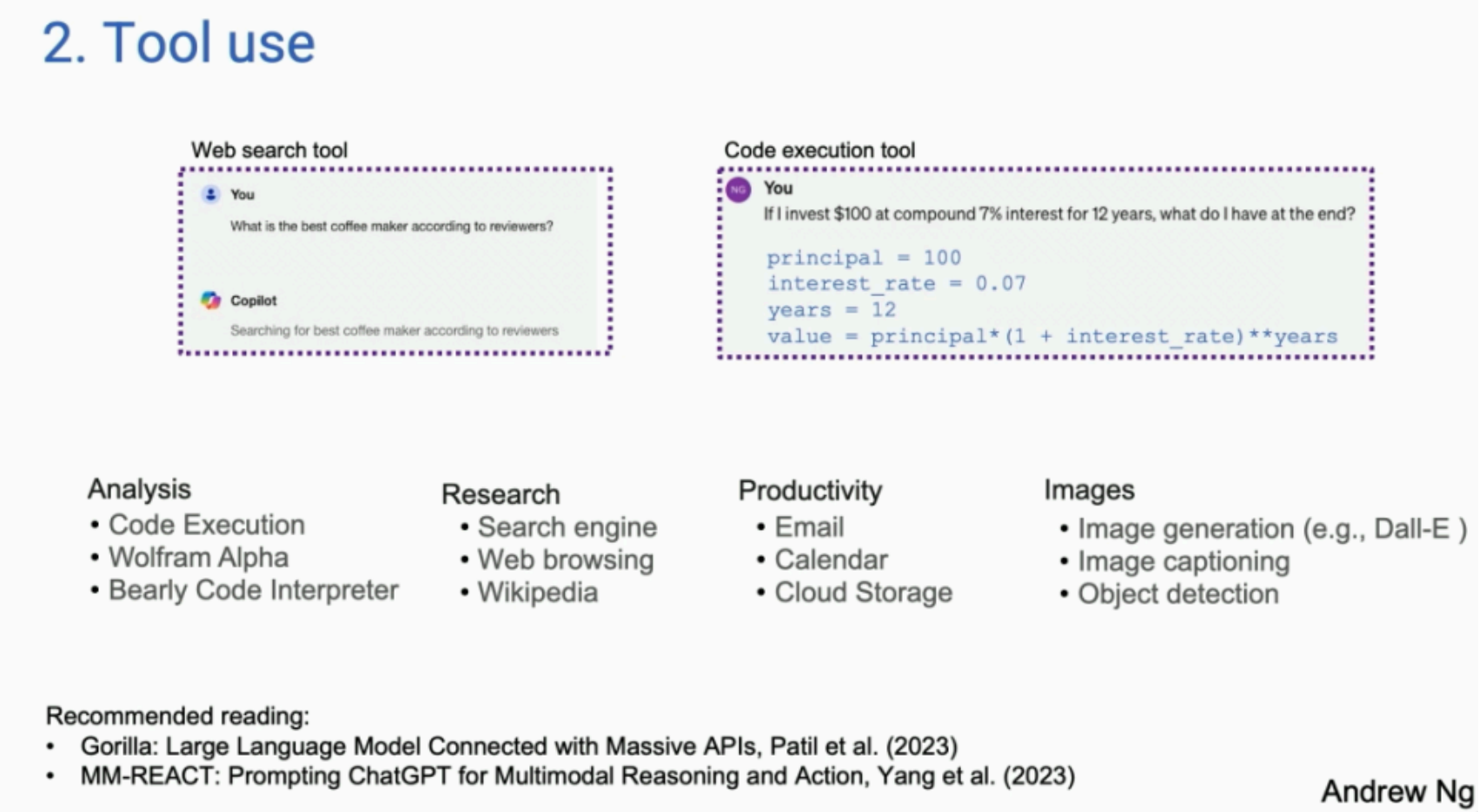
不保证，但它经常足够有效，对很多应用来说值得一试，如果你进行运行单元测试，如果它未通过单元测试，那么为什么会未通过单元测试？有这样的对话可能会弄明确为什么未通过单元测试。所以尝试改变一些东西，可能会得到第三版本。

顺便说一下，对于那些想要了解更多这些技术的人，我对他们感到非常兴奋，对于每一个的四个部分中，我都在底部有一个推荐阅读部分，那里，包含了更多的参考资料。

再次来说，多代理系统，我描述的是一个单独的代码代理，你提示它进行自我对话。这个想法的一个自然就可以是，不是一个单独的代码代理可以，你有两个代理，其中一个是代码Agent，另一个是批评Agent。这些可以是相同的基础LLM，但以不同的方式进行提示。我们说一个，你是专家级的代码编写者，对吧？写代码。另一个说，你是专家级别的代码审查者，审查可能的代码。

这个工作流程实际上很容易实现。我认为这是一种通用的技术，对工作流程而言。这将显着提高LLM的性能。

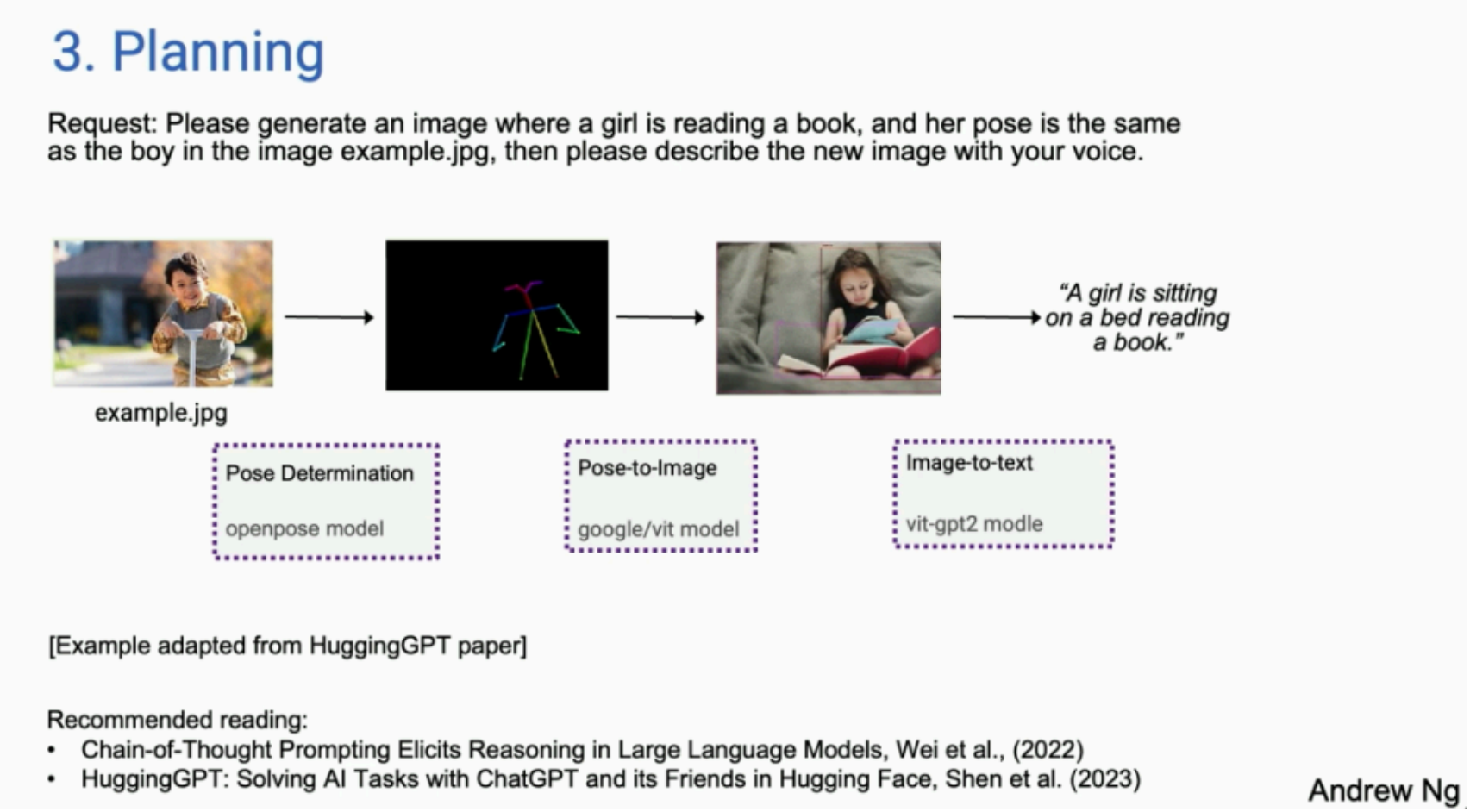
2.工具使用



其次的设计模式是很多人都见过的，基于LLM的系统使用的工具，左边是一个来自Copilot的截图。右边是我从GPT-4中提取的东西，但今天的LLM，如果你问的话，什么是网上搜索中最好的咖啡机，某些对问题，LLM会生成代码并运行代码。事实证明，有很多不同的工具被许多不同的人用于分析、获取信息、采取行动、个人生产力。

早期的工作转向使用，本来是在计算机视觉社区。因为在LLM之前，他们无法处理图像。所以唯一的选择是生成一个可以读取图像的函数调用，比如生成一个图像或进行对象检测等。如果你实际上看看文献，很有趣的是，很多在使用方面的工作似乎都起源于视觉领域，因为在GPT-4等出现之前，LLM对图像是盲目的，这就是使用，并扩展了LLM可以做的事情。

3.规划



然后是规划，对于那些没有大量玩过规划算法的人来说，我觉得很多人在谈论 ChatGPT 时刻，你会惊叹，从未见过这样的东西。我认为你还没有使用规划算法。人对人工智能代理有种令人惊讶的感觉。

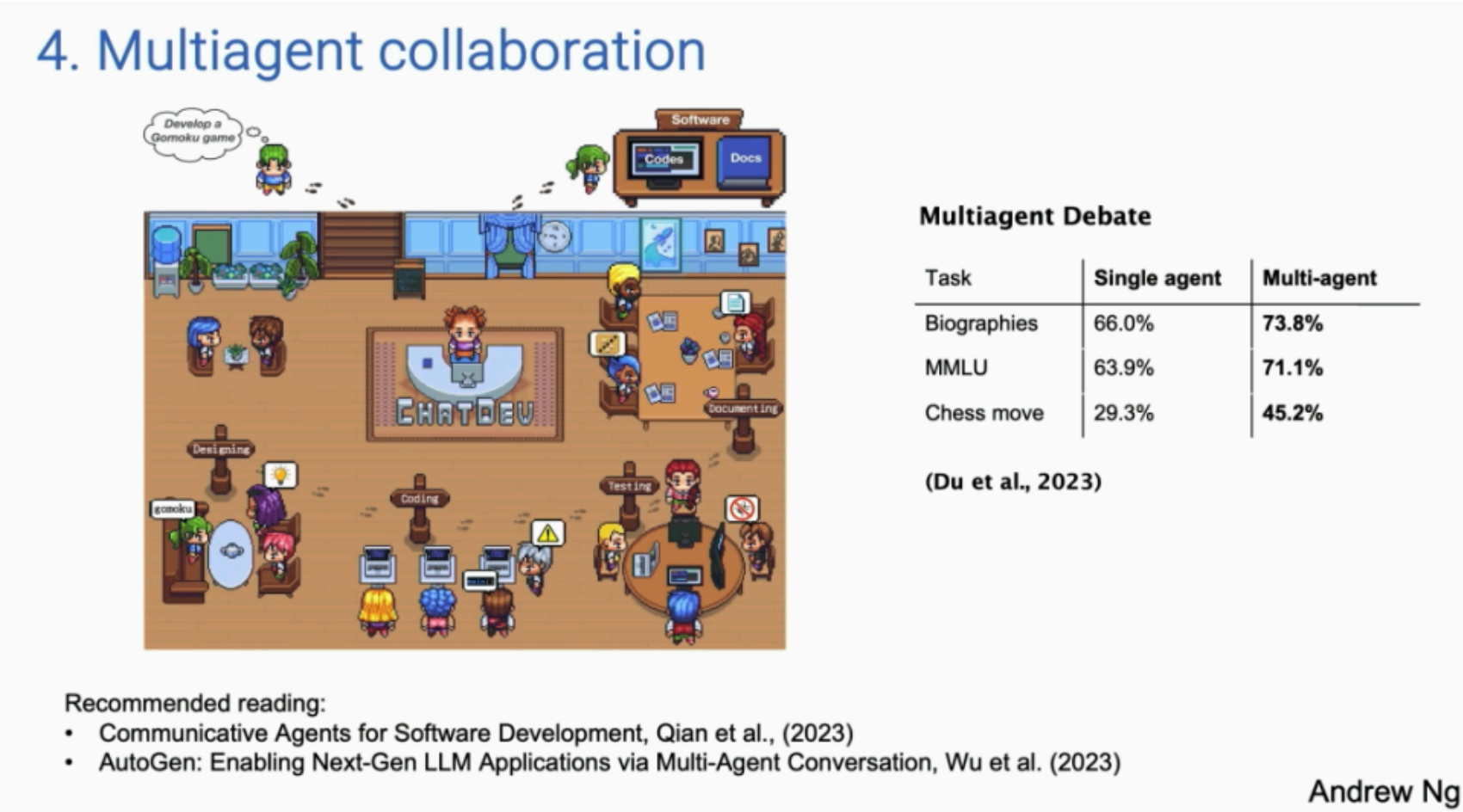
我无法想象 AI Agent 能做得这么好。我进行过实时演示，其中有一些失败了，AI Agent 绕过了这些失败。我实际上遇到了惊人的这样的情况，是的，我无法相信我的 AI系统刚刚自主地做到了那一点。

但是一个从 HuggingGPT 论文中改编的例子，你说，请生成一张图片，图片中的女孩在读书中，并且与图像示例 dot jpeg 中的男孩姿势相同，请用语音描述新图像。所以给出这样一个例子，今天有人工智能代理，你可以决定首先需要做确定男孩的姿势。然后，找到正确的模型，可能在HuggingFace上提取姿势。接下来需要找到姿势图像模型来合成一张女孩的图片，遵循指令。然后使用图像检测，最后使用文本到语音。

今天我们实际上有代理，我不想说它们工作非常可靠，它们有点选择性。它们并不总是工作，但当它工作时，实际上相当不可思议，但有了代理性循环，有时你也可以从早期开始的失败中过来了。所以我发现我已经在使用研究代理了。所以我的一些工作，一部分研究，但我不觉得，自己去谷歌搜索并花很长时间。我应该发送给研究代理，几十年后回来看看发现它是什么，有时有效，有时没有，但已经是我个人工作流程的一部分了。

4.多智能体协作

最后一个设计模式，多Agent合作，这个，听起来很有趣，但是它的效果比你想象的要好。左边是一篇叫ChatDev的论文的截图，这完全是开源的，实际上是开源的。你们很多人都看到了，闪亮的社交



媒体发布的演示，ChatDev 是开源的，在我的笔记本电脑上运行。ChatDev 是一个多代理系统的示例，你可以提示一个 LLM 有时表现得像软件工程公司的首席执行官，有时像设计师，有时像产品经理，有时像测试人员。

通过提示LLM告诉它现在你是CEO，现在你是软件工程师，他们合作，进行扩展的对话，以至于如果你告诉它，请开发一个游戏，开发一个多个游戏，他们实际上会花费几千编写代码，测试，迭代，并最终生成一个出人意料的复杂程序。

这种多代理合作听起来可能有些奇特，但实际上它的效果比你想象的要好。这不仅仅是因为这些代理之间的合作能够带来更丰富和多样的输入，而且因为它能够模拟出一个更加接近真实工作环境的场景，其中不同角色和专业知识的人员为了共同的目标而努力。这种方式的强大之处在于它能够让LLM不仅仅是执行单一任务的工具，而是成为一个能够处理复杂问题和工作流程的协作系统。

这种方法的潜在价值巨大，因为它为自动化和提升工作流程的效率提供了新的可能性。例如，通过模拟一个软件开发团队的不同角色，一个企业可以自动化某些开发任务，从而加快项目的速度同样，这种多代理合作方式也可以考察其他领域，如内容创作、教育和培训、以及策略规划等，进一步拓宽LLM在各个行业的应用范围。

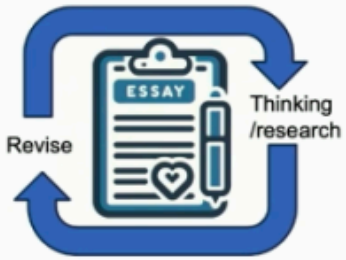
总结

Conclusion

The set of tasks that AI can do will expand dramatically because of agentic workflows.

We have to get used to delegating tasks to AI agents and patiently wait for a response.

Fast token generation is important. Generating more tokens even from a lower quality LLM can give good Results.



If you're looking forward to running GPT-5/Claude 4/Gemini 2.0 (zero shot) on your application, you might already be able to get similar performance with agentic reasoning on an earlier model.

Andrew Ng

总之，通过Agent Reflection、规划与多Agent合作等设计模式，我们不仅能够提升LLM的性能，还能够拓展它们的应用领域，使它们成为更加强大和灵活的工具。随着这些技术的不断发展和完善，我们期待未来的AI Agent能够在更多的场景中发挥关键作用，为人们带来更加智能和高效的解决方案。

它并不总是有效。我用过它。有时它不起作用，有时它令人惊讶，但这个技术确实在变得更好。还有一个设计模式，事实证明，多代理辩论，即不同Agent之间的辩论，例如，可以让ChatGPT和Gemini进行辩论，这实际上也能带来更好的表现。

因此，让多个模拟的空气代理一起工作也是一个强大的设计模式。综上所述，我认为这些是我见过的模式。我认为，如果我们能够使用这些模式，我们中的许多人很快就能实现实践上的提升。我认为，Agent推理设计模式非常重要。

这是我的摘要幻灯片。我预计，因为特工的工作流程，人工智能能够完成的任务将在今年大幅扩展。一个实际上让人难以习惯的事情是，当我们向 LLM 发出提示时，我们希望立即实际上，十年前当我在Google讨论我们称大框搜索时，输入长提示

的原因之一，我未能成功推动的原因之一是因为当你进行网络搜索时，你希望在半个时间内得到响应，对吧？ 这就是人性，即时抽取，即时反馈。

对于许多代理工作流程，我认为我们需要学会将任务委托给人工智能代理，并耐心等待数千，甚至几小时才回复，但就像我见过的新手经理将任务委托给某人，然后五分钟后面检查一样，对吧？这不是生产性的。

我认为我们也需要学会这样做，与我们的一些AI Agent打交道，虽然这很难。我以为我听到了一些损失。然后一个重要的趋势是，快速代币生成器非常重要，因为在这些Agent工作流程中，我们不断迭代。所以LLM为LLM生成代币，能够以远超任何人阅读速度的速度生成代币是四分之一的。

我认为，即使是来自稍微低质量的LLM，快速生成更多的代币也可能与来自更好的LLM的慢代币相比，获得好的结果。这可能有点争议，因为它可能让你更多次地绕这个循环，就像我在第一张幻灯片上展示的 GPDC 和代理架构的结果一样。

坦率地说，我非常期待 Claude5 和 Claude4，GPT-5 和 Gemini 2.0 以及所有这些你们正在构建的精彩模型。我部分感觉，如果你期待在 GPT-5 上运行你的东西，零次射击，你可能通过特工推理，确实会在某些应用上获得比你想象的更接近水平的表现，但在早期模型上，我认为这是一个重要的趋势。

老实说，通往 AGI 的道路感觉就像是一次旅程而不是一个目的地，但我认为这种 Agent 工作流程可能会帮助我们在这个非常长的旅程上向前迈出一步。